

Git Guild 详细设计文档

1. 文档目的

本文件整合 P3 阶段详细设计成果，包括核心类图、SOLID 检查、API 规范、数据库设计和 AI 协作审查记录。P3 详细设计基于 P1 需求规格和 P2 架构设计展开，目标是在实现前明确对象模型、接口边界和数据结构。

2. 设计依据

来源	文件	作用
P1 需求分析	docs/P1/Git Guild 需求规格说明书.pdf	明确核心用户、业务场景和用例
P2 架构设计	docs/P2/架构设计文档.md	明确总体架构、模块和技术选型
P2 模块划分	docs/P2/模块划分与职责边界说明.md	明确 P0 / P1 / P2 优先级
P3 阶段要求	/home/wuren/P3-详细设计.pdf	明确本阶段交付物和验收标准

3. 系统模块与优先级

模块	优先级	P3 设计覆盖内容
用户与认证	P0	用户类、认证服务、注册登录 API、用户表
代码托管底座适配	P0	适配器接口、GitHub/Gitea 实现、仓库 API、仓库表
任务与悬赏	P0	任务、分类、标签、接取、任务 API、任务相关表
推荐匹配	P0	推荐策略接口、推荐结果、推荐 API、推荐结果表
提交与审核反馈	P0	提交、维护者审核、管理员审核、审核 API、审核表
新手引导与项目理解	P0	项目指引类、项目指引 API、项目指引表
通知	P0	通知实体、通知发送接口、通知偏好、通知表
成长激励	P1	基础成长档案、XP 流水、贡献记录

说明：推荐匹配是 P0 核心能力，不作为扩展功能处理。成长激励在本阶段保留轻量模型，用于支撑审核通过后的基础反馈。

4. 核心类设计

核心类图已拆分为关系图和类成员说明：

文件	内容
核心类图.md	按用户与外部支撑、任务核心、提交审核与成长、推荐与通知拆分的 Mermaid 类图
类成员说明.md	每个核心类的关键属性、方法和标注说明
SOLID检查清单.md	AI 原始类图的 SOLID 违规记录和修正方案

核心设计结论：

- 实体类只表达领域状态和少量领域判断，不承担外部 API 调用、登录签发、通知发送等基础设施行为。
- 外部代码托管差异通过 `CodeHostAdapter` 抽象隔离，业务模块不直接依赖 GitHub 或 Gitea API。
- 推荐规则通过 `RecommendationStrategy` 扩展，保证推荐作为核心模块仍具备后续演进空间。
- 通知通道通过 `NotificationSender` 隔离，避免通知服务同时绑定站内通知和邮件实现。

5. 设计模式应用

设计模式	应用位置	解决的问题
适配器模式	<code>CodeHostAdapter</code> 、 <code>GitHubImportAdapter</code> 、 <code>GiteaAdapter</code>	屏蔽 GitHub 与 Gitea API 差异， 稳定业务模块依赖
策略模式	<code>RecommendationStrategy</code> 及其实现类	支持按技术栈、成长阶段、历史 贡献等规则扩展推荐算法
策略 / 端口模式	<code>NotificationSender</code> 及其实现类	支持站内通知、邮件通知等投递 通道扩展

如果不使用这些模式，外部平台调用、推荐规则和通知渠道会散落在业务服务中，后续新增平台或推荐规则时需要修改大量已有代码，违反开闭原则和依赖倒转原则。

6. API 设计

API 规范采用 RESTful 风格，基础路径统一为 `/api/v1`。所有需要登录的接口通过 `Authorization: Bearer <JWT_TOKEN>` 传递认证信息。

设计文件	覆盖内容
API规范/API规范总览与AI审查.md	API 文档目录、P3 要求覆盖情况、AI 审查
API规范/	各模块详细接口规范

P3 要求的 6 类核心接口均已覆盖：

P3 要求接口	当前接口
用户注册 / 登录	<code>POST /api/v1/auth/register</code> 、 <code>POST /api/v1/auth/login</code>
发布需求	<code>POST /api/v1/quests</code>
浏览需求列表（含筛选）	<code>GET /api/v1/quests</code>
接单	<code>POST /api/v1/quests/{questId}/assignments</code>
查看订单详情	<code>GET /api/v1/quests/{questId}</code>
提交评价	<code>POST /api/v1/submissions/{submissionId}/reviews</code>

统一接口约束：

成功和失败响应均包含 `code`、`message`、`timestamp`、`traceId`。

角色权限由后端校验，不能依赖前端页面隐藏。

接取人、审核人等身份字段从 JWT 解析，前端不传可信 `userId`。

Webhook、审核、同步等接口需要考虑幂等和重复调用。

7. 数据库设计

数据库设计见 `数据库设计.md`，包含 ER 图、核心实体说明、MySQL 建表 SQL、索引设计、隐私处理、第三范式检查和 AI 辅助审查记录。

数据库设计原则：

平台账号独立存储，不依赖 GitHub OAuth。

仓库、Issue、PR 只保存平台需要的元数据，不自研底层 Git 存储。

任务分类、标签、接取记录、审核记录独立建表，保证任务流程可追踪。

推荐结果、XP 流水、贡献记录、通知记录保留历史，便于解释和展示。

密码只保存哈希值，Webhook 密钥和外部访问令牌不明文进入业务库。

8. 关键业务流程落地

8.1 任务发布与管理员审核

维护者通过任务 API 基于仓库和 Issue 创建任务。

任务进入 `PENDING_ADMIN_REVIEW` 状态。

管理员调用管理员审核 API，决定发布、退回或下架。

审核通过后任务进入 `PUBLISHED`，可被任务大厅展示和推荐模块读取。

8.2 任务筛选、推荐与接取

用户可通过任务列表 API 主动筛选任务。

推荐模块根据技术栈、成长阶段和任务特征生成推荐结果。

用户接取任务时，后端校验任务状态并创建 `quest_assignments` 记录。

接取成功后发布通知事件，任务状态进入进行中。

8.3 成果提交与审核反馈

用户完成开发后通过代码支撑平台创建 Commit / PR。

用户提交成果并关联 PR。

维护者或管理员审核提交，生成 `review_records` 和 `review_items`。

审核通过后触发任务完成、通知发送和成长记录更新。

9. AI 辅助实验结论

实验项	主要发现	人工修正
类图生成	AI 容易把实体、服务和外部平台调用混在一个类中	拆分实体、服务、适配器和策略接口
SOLID 检查	原始设计存在 15 处 SOLID 风险	记录在 <code>SOLID检查清单.md</code> ，并完成修正类图
API 生成	AI 草案命名不一致，错误码、权限和参数校验不足	建立 API 总览与模块化规范，补充认证、错误码和失败响应
数据库审查	AI 容易忽略索引、历史记录、隐私字段和多对多关系拆分	补充索引设计、历史记录表、密码哈希要求和关联表

10. 交付物清单

序号	交付物	文件位置	状态
1	类图	<code>docs/P3/核心类图.md</code>	已完成
2	类成员说明	<code>docs/P3/类成员说明.md</code>	已完成
3	SOLID 检查清单	<code>docs/P3/SOLID检查清单.md</code>	已完成
4	API 规范文档	<code>docs/P3/API规范/</code>	已完成
5	API AI 审查记录	<code>docs/P3/API规范/API规范总览与AI审查.md</code>	已完成
6	ER 图与建表 SQL	<code>docs/P3/数据库设计.md</code>	已完成
7	详细设计整合文档	<code>docs/P3/详细设计文档.md</code>	已完成
8	AI 协作反思日志 #3	<code>docs/P3/AI 协作反思日志 #3.md</code>	已完成

11. 结论

P3 详细设计已覆盖课程要求中的类图、SOLID 检查、API 规范、数据库设计、AI 辅助审查和整合文档。当前设计坚持 P2 的模块化单体架构，以任务协作闭环和推荐匹配为核心，同时对外部代码托管底座、推荐策略、通知通道等变化点保留扩展接口。